Multi-Agent Graph-Attention Communication

Yaru Niu
Georgia Institute of Technology
Atlanta, Georgia
yaruniu@gatech.edu

Rohan Paleja
Georgia Institute of Technology
Atlanta, Georgia
rpaleja3@gatech.edu

Matthew Gombolay
Georgia Institute of Technology
Atlanta, Georgia
matthew.gombolay@cc.gatech.edu

ABSTRACT

High-performing teams learn effective communication strategies to judiciously share information and reduce the cost of communication overhead. Within multi-agent reinforcement learning, synthesizing effective policies requires reasoning about when to communicate, whom to communicate with, and how to process messages. We propose a novel multi-agent reinforcement learning algorithm, Multi-Agent Graph-attentIon Communication (MAGIC), with a graph-attention communication protocol in which we learn 1) a Scheduler to help with the problems of when to communicate and whom to address messages to, and 2) a Message Processor using Graph Attention Networks (GATs) with dynamic graphs to deal with communication signals. The Scheduler consists of a graph attention encoder and a differentiable attention mechanism, which outputs dynamic, differentiable graphs to the Message Processor, which enables the Scheduler and Message Processor to be trained end-to-end. We evaluate our approach on a variety of cooperative tasks, including Google Research Football. Our method outperforms baselines across all domains, achieving $\approx 10\%$ increase in reward in the most challenging domain. We also show MAGIC communicates 23.2% more efficiently than the average baseline, is robust to stochasticity, and scales to larger state-action spaces. Finally, we demonstrate MAGIC on a physical, multi-robot testbed.

KEYWORDS

Multi-Agent Reinforcement Learning, Graph-based Communication, Multi-Agent Communication

ACM Reference Format:
Yaru Niu, Rohan Paleja, and Matthew Gombolay. 2021. Multi-Agent Graph-Attention Communication. In *Proc. of the 20th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2021), London, UK, May 3–7, 2021*, IFAAMAS, 9 pages.

1 INTRODUCTION

Communication is a key component of swarm intelligence, which enables the agents to convey information and cooperate to collectively achieve shared goals [26, 31]. In high-performing human teams, human experts judiciously choose when to communicate and whom to communicate with, communicating only when beneficial [7, 30, 40]. Each team member exhibits the role of a communicator and message receiver, relaying valuable information to the right teammates and incorporating received information effectively.

In this paper, we propose Multi-Agent Graph-attentIon Communication (MAGIC), a novel graph communication protocol that determines "when" and "whom" with to communicate via an end-to-end framework. We set a new state-of-the-art in communication-based multi-agent reinforcement learning (MARL) by modeling the topology of interactions among agents (the local and global characterization of connections between agents [1]) as a dynamic directed graph that accommodates time-varying communication needs and accurately captures the relations between agents. Our proposed framework emulates the features of an effective human-human team through its key components, 1) the Scheduler, which helps each agent to decide when it should communicate and whom it should communicate with, and 2) the Message Processor, which integrates and processes received messages in preparation for decision making. We find MAGIC produces high-performance, cooperative behavior through its efficient communication protocol.

There has been recent success in generating high-performing cooperative behavior using MARL in Multiplayer Online Battle Arena (MOBA) games such as StarCraft II and Dota II [4, 5, 32, 42]. MARL seeks to enable agents to share information to improve team performance [8, 9, 27, 37, 38, 43]. However, most prior work in MARL fails to capture the complex relations among agents, leading to low-performance and inefficient communication. While [37] and [15] are able to efficiently decide when to broadcast messages, agents will broadcast these messages to all other agents without targets, resulting in wasteful communication. Even with targeted communication [8], failure to assess when to communicate results in poor performance, as we display in Section 6. However, determining when to communicate and whom to communicate with is not enough. Selectively utilizing received messages rather than averaging over messages [37, 38] can significantly improve performance. Yet, none of these methods simultaneously address "when" and with "whom", and "how" to communicate while modeling agent interaction topology.

Our communication protocol, MAGIC, utilizes a Scheduler consisting of a graph attention encoder and a differentiable hard attention mechanism to decide when to communicate and whom to communicate with. This information is encoded within a directed graph, allowing us to represent the interaction among agents precisely. The Message Processor consisting of a Graph Attention Network (GAT), utilizes received messages and the directed graph to intelligently and efficiently process messages. The encoded messages are then used in each agent's policy, leading to high-performance cooperation and efficient communication, as shown in Section 6. We provide the following contributions:

(1) Develop a novel graph-attention communication protocol for MARL that utilizes 1) a Scheduler to solve the problems of when to communicate and whom to address messages to, and 2) a Message Processor using GATs with dynamic directed graphs to integrate and process messages.

Proc. of the 20th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2021), U. Endriss, A. Nowé, F. Dignum, A. Lomuscio (eds.), May 3–7, 2021, London, UK. © 2021 International Foundation for Autonomous Agents and Multiagent Systems (www.ifaamas.org). All rights reserved.

(2) Enable GATs in the Message Processor to deal with differentiable graphs, which is not supported by standard GATs. In this way, the framework is completely differentiable and can be trained in an end-to-end manner.
(3) Outperform prior methods across three domains, including the Google Research Football environment, achieving a 10.1% increase in reward. Further, MAGIC learns to communicate 23.2% more efficiently than the average baseline. These results set a new state-of-the-art in MARL.
(4) Demonstrate our algorithm on physical robots in a 3-vs.-2 soccer scenario on the Robotarium platform [28], a physical, multi-robot testbed.

2 RELATED WORK

In multi-agent settings, each agent observes other agents as part of the environment, causing the environment to appear dynamic and non-stationary. Further difficulty arises due to the issue of credit assignment, where it is difficult for each agent to deduce its own contribution to the team's success (especially when there are only global rewards). To solve these multi-agent challenges, many researchers in MARL [6] have pursued centralized training and decentralized execution. Further extensions allow agents to exchange messages during execution, allowing for increased performance. Here, we present recent work in MARL including those with a centralized critic, multi-agent communication, and graph neural networks.

MARL with Centralized Critic – Some works extend variants of actor-critic algorithms to multi-agent settings and learn decentralized policy through centralized critics without explicit communication channels [10, 11, 23]. MADDPG [23] is a MARL framework based on Deep Deterministic Policy Gradient (DDPG), and can be applied in both cooperative and competitive scenarios. COMA [10] extends on-policy actor-critic and proposes a counterfactual baseline to address the credit assignment problem. While these works present critical improvements in the field of multi-agent reinforcement learning, the ability to communicate and process information allows for much-increased performance. As we show in Section 6.4, the ability to communicate results in a 88.9% performance gain for our method.

MARL with Communication – Recent works have enabled agents to communicate and exchange messages during execution. Differentiable Inter-Agent Learning (DIAL) [9] builds up limited-bandwidth differentiable discrete communication channels among agents. CommNet [38] extends to a continuous communication protocol designed for fully cooperative tasks. Agents receive averaged encoded hidden states from other agents and use the messages to make decisions. However, utilizing a sum or average of messages results in some information loss. IC3Net [37] (an extension of CommNet) uses a gating mechanism to enable the agents to decide when to communicate, and thus is amenable to competitive scenarios. However, both IC3Net and CommNet process messages with a simple average. The proper integration of these messages is critically important for communication, as displayed by the performance of our results in Section 6. TarMAC [8] achieves targeted communication with a signature-based soft-attention mechanism. The integrated signal for each agent is the weighted mean of values generated by all agents. This work does not explicitly consider when to communicate and whom to send messages or the topology of agent interactions, which can help save communication resources and process messages more elaborately and efficiently. ATOC [14], employs an attention mechanism to decide if an agent should communicate in its observable field. SchedNet [15] proposes a weight-based scheduler to pick agents who should broadcast their messages. However, both ATOC and SchedNet have to manually configure the size of a communication group, a difficult-to-determine hyperparameter. Our communication protocol is able to intelligently decide when to communicate and whom to communicate with through a graph-attention based Scheduler, and effectively process received messages, resulting in a large performance gain ($\approx$ 33% average increase in reward in our most difficult domain) compared to several MARL baselines.

MARL using Graph Neural Networks Graph Neural Networks (GNNs) are powerful tools for learning from data with graph structures [34, 44, 47]. To model the interactions between agents, MARL has utilized GNNs to allow for a graph-based representation [13, 19, 24, 35]. DGN [13] represents the multi-agent environment as a graph and employ multi-head dot-production attention as the convolutional kernel to extract relational features between agents. MAGNet [24] learns multi-agent policies in the Pommerman game by utilizing a relevance graph and message passing mechanism. The graphs are generated based on heuristic rules, and the graph structure in each time step stays fixed. [35] learns a hierarchical topology of the communication structure dynamically by electing central agents. A GNN is then used to compute the messages further update overall state perception. HAMA [29] designs a hierarchical graph attention network to model the hierarchical relationships between agents in both cooperative and competitive scenarios. G2ANet [22] combines a hard-attention and a soft attention mechanism to dynamically learns interactions between agents. In this work, we modify standard graph attention networks to be compatible with a differentiable directed graph, allowing us to represent the interactions among agents more accurately during communication.

We improve upon prior frameworks by utilizing a Scheduler to solve the problems of when to communicate and whom to address messages to, and a Message Processor using GATs with dynamic directed graphs to integrate and process messages. In this way, we achieve efficient and targeted message sending and high-performance message comprehension.

3 PRELIMINARIES

In this section, we introduce Partially Observable Markov Games, Policy Gradient, and Graph Neural Networks.

3.1 Partially Observable Markov Game

A Markov Game [21] is the multi-agent version of Markov Decision Process (MDP). We are primarily concerned with a partially observable Markov game. A partially observable Markov game (POMG) for N agents can be defined by a set of global states, S, a set of private observations for each agent, $O_1, O_2, \ldots, O_N$, a set of actions for each agent, $A_1, A_2, \ldots, A_N$, and the transition function, $T : S \times A_1 \times \ldots \times A_N \mapsto S$. In each time step, agent i chooses action, $a_i \in A_i$, obtains reward as a function of state, S, and its action

$r_i : S \times A_i \mapsto \mathbb{R}$, and receives a local observation $o_i : S \mapsto O_i$. The initial state is defined by a initial state distribution ρ. Agent i aims to maximize its discounted reward $R_i = \sum_{t=0}^{T} \gamma^t r_i^t$, where $\gamma \in [0, 1]$ is a discounted factor. Our work is based on the framework of POMG augmented with communication.

3.2 Reinforcement Learning: Policy Gradients

The Policy Gradient method is widely used in reinforcement learning (RL) tasks to perform gradient ascent on the agent policy parameters, θ, to optimize the total discounted reward, $J(\theta) = \mathbb{E}_{s\sim p^{\pi}, a\sim\pi_\theta}[R]$ ([2, 20, 36]). Following [39, 46], the policy gradient is displayed in Equation 1.

$$\nabla_\theta J(\theta) = \mathbb{E}_{s\sim\rho^{\pi}, a\sim\pi_\theta}\Big[\sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t) R_t\Big] \tag{1}$$

Here, ρ^{π} denotes the state distribution, π_θ denotes the policy distribution, and $R_t = \sum_{t'=t}^{T} \gamma^{t'-t} r(s_{t'}, a_{t'})$. It is helpful to use the advantage function, $A^{\pi}(s_t, a_t) = R_t - V(s_t)$, to decrease the variance of the estimated policy gradient while keeping the estimate unbiased, where the learned value function, $V(s_t)$, is used as the baseline [3, 17, 45].

3.3 Graph Neural Networks

In Graph Neural Networks (GNNs), each GNN layer computes the node representation by message passing, where each node aggregates the feature vectors from its neighboring nodes in the graph at the previous layer. Graph Convolutional Networks (GCN) [16, 33] can be seen as a variant of vanilla GNNs. The update rule for node representations by a GCN layer is displayed in Equation 2.

$$h_i^{(l)} = \sigma\left(\sum_{j\in N_i} \frac{1}{\sqrt{d_i d_j}} (h_j^{(l-1)} W^{(l)})\right) \tag{2}$$

Here, $h_i^{(l)}$ represents the features of node i, at layer l. N_i denotes the set of neighboring nodes of node i in the graph, $d_i = |N_i|$ denotes the degree of node i, $W^{(l)}$ is a learnable weight matrix for layer l, and $\sigma(\cdot)$ denotes a nonlinear activation function. In this paper, we enable our GATs within the Message Processor to function with differentiable graphs (described in Section 4.3), allowing us end-to-end training.

4 METHOD

In this section, we introduce our proposed Multi-Agent Graph-attentIon Communication protocol, MAGIC. We consider a partially observable setting of N agents, where agent i receives local observation, o_i^t, at time, t, containing local information from the global state, S. The agent, i, learns a communication-based policy, π_i, to output a distribution over actions, $a_i^{(t)} \sim \pi_i$, at each time step, t. Here, we present an overview of our framework, the description of our protocol's key components (i.e., the Scheduler and Message Processor), and our training procedure.

4.1 Overview

Our proposed graph-attention communication protocol is displayed in Figure 1. At time step, t, the observation for each agent, o_i^t, is

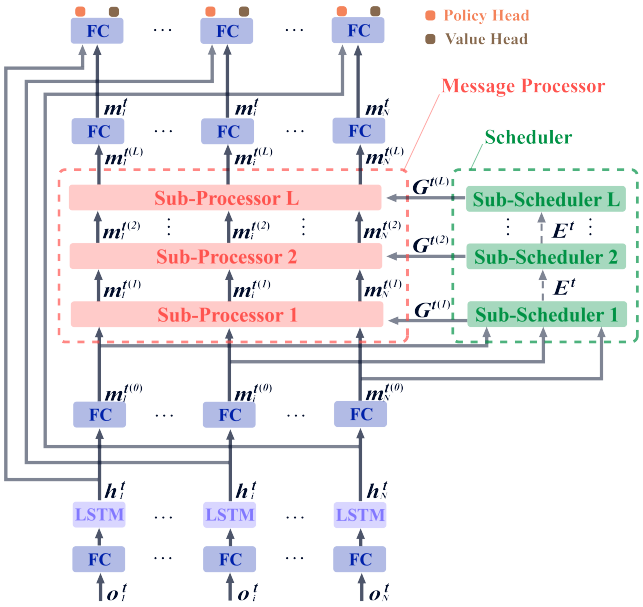


Figure 1: This figure displays the framework of our multi-agent graph-attention communication protocol.

first encoded using an agent-specific fully-connected layer (FC). The encoded observation is passed into an agent-specific LSTM cell to generate a hidden state, h_i^t, as shown in Equation 3.

$$h_i^t, c_i^t = LSTM(e(o_i^t), h_i^{t-1}, c_i^{t-1}) \tag{3}$$

In this equation, c_i^t is the cell state for agent, i, at time step, t, and $e(\cdot)$ is a fully-connected layer acting as an encoder for the observation. The hidden state, h_i^t, is then encoded as a message, $m_i^{t(0)} = e_m(h_i^t)$, through the encoder, $e_m(\cdot)$ (a fully-connected layer). Here, the exponent notation for the message, $m_i^{t(0)}$, denotes that message is for agent i, and is prior to any message aggregation or processing. We refer to this stage, where the message has not been processed, as round 0, giving the exponent notation, $t(0)$.

As shown in Figure 1, we define the function module to help agents decide whom to send messages at each time step as the "Scheduler" and define the function module to process messages as "Message Processor." The Scheduler and the Message Processor may include multiple sub-schedulers and sub-processors, respectively. Prior work has termed the procedure of processing messages for multiple iterations as multi-round communication [8]. As multi-round communication has been shown to improve performance, our protocol supports L rounds of communication, where $L \in \mathbb{N}$. A round of communication, l, is defined as a forward pass through a sub-scheduler and sub-processor. As shown in Figure 1, the encoded messages, $m_i^{t(0)}$ are passed into Sub-Scheduler 1 and Sub-Processor 1 (i.e., the sub-scheduler and sub-processor at round 1).

The Sub-Scheduler l (at round, $l \in L$) will output an adjacency matrix, $G^{t(l)}$. $G^{t(l)}$ is a directed graph that indicates the targeted receivers for each agent at time step, t. $G^{t(l)}$ is utilized by the Sub-Processor, l, to produce a set of integrated messages, $\{m_i^{t(l)}\}_1^N$,

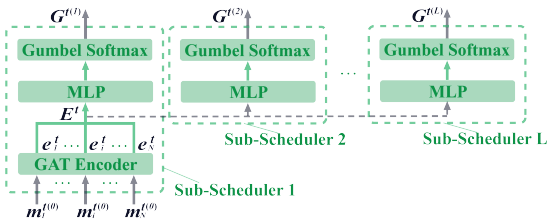


Figure 2: This figure displays the details and components of the Scheduler.

where $m_i^{t(l)}$ is the integrated message for agent, i, at time step, t. The integrated messages for each agent, i, can be incorporated into agent i's policy (in the case where we are on the last round of communication, $l = L$) or be further processed by more rounds of communication ($l < L$). If the messages are to be further processed, the set of messages outputted from round l, $\{m_i^{t(l)}\}_1^N$, are passed into the Sub-Scheduler $l+1$ and the Sub-Processor $l+1$, producing adjacency matrix $G^{t(l+1)}$ and messages $\{m_i^{t(l+1)}\}_1^N$ respectively.

The message outputted from the Message Processor, $m_i^{t(L)}$, for agent, i, is encoded through a fully-connected layer, $e'_m(\cdot)$, to produce an intelligently integrated message, $m_i^t = e'_m(m_i^{t(L)})$. m_i^t is concatenated with the hidden state, h_i^t, to produce the input feature to the policy head and the value head. The policy head is a fully-connected layer followed by a softmax function. We sample the action for the i-th agent at time step, t, from the policy output distribution: $a_i^t \sim \pi_i(a_i^t|o_i^t)$. The value head is a single fully-connected layer and serves as a baseline function for our MARL algorithm.

4.2 The Scheduler

The Scheduler decides when each agent should send messages and whom each agent should address messages to, as shown in Figure 2. As a black-box, the Scheduler takes as input the encoded messages, $\{m_i^{t(0)}\}_1^N$, and outputs the directed graphs, $\{G^{t(l)}\}_1^L$, as represented in $f_{Sched}(\cdot)$ shown in Equation 4.

$$\{G^{t(l)}\}_1^L = f_{Sched}\left(m_1^{t(0)}, \cdots, m_N^{t(0)}\right) \tag{4}$$

As noted in Section 4.1, the Scheduler consists of L Sub-Schedulers, each producing an adjacency matrix $G^{t(l)}$. A Sub-Scheduler consists of a GAT encoder and a hard attention mechanism that uses a multi-layer perceptron (MLP) and a Gumbel Softmax function [12]. The GAT encoder helps encode local or global information for an agent efficiently, and it is only used in the first Sub-Scheduler. We adopt the same form of GATs as proposed in [41], where the attention mechanism is expressed in Equation 5.

$$\alpha_{ij}^S = \frac{exp\left(LeakyReLU\left(a_S^T[W_S m_i^{t(0)}||W_S m_j^{t(0)}]\right)\right)}{\sum_{k\in N_i^t\cup i} exp\left(LeakyReLU\left(a_S^T[W_S m_i^{t(0)}||W_S m_k^{t(0)}]\right)\right)} \tag{5}$$

Here, $LeakyReLU(\cdot)$ is the LeakyReLU nonlinearity [25], $a_S \in R^{D'}$ represents a weight vector for the sub-scheduler, $N_i^t \cup i$ denotes the set of neighboring agents for the i-th agent, including agent i, at time step, t, and $W_S \in R^{D'\times D}$ represents a weight matrix for the sub-scheduler, where D' and D refer to the output feature cardinality and message cardinality, respectively. The node features of each agent are obtained through Equation 6.

$$e_i^t = ELU\left(\sum_{j\in N_i^t\cup i} \alpha_{ij}^S W_S m_j^{t(0)}\right) \tag{6}$$

Here, $ELU(\cdot)$ is the Exponential Linear Unit (ELU) function. After concatenating the node feature vectors pairwise, supposing $e_{ij}^t = (e_i^t||e_j^t)$, we can obtain a matrix $E^t \in R^{N\times N\times 2D}$:

$$E^t = \begin{pmatrix} e_{11}^t & e_{12}^t & \cdots & e_{1N}^t \\ e_{21}^t & e_{22}^t & \cdots & e_{2N}^t \\ \vdots & \vdots & \ddots & \vdots \\ e_{N1}^t & e_{N2}^t & \cdots & e_{NN}^t \end{pmatrix} \tag{7}$$

Setting E^t as the input to an MLP followed by a Gumbel Softmax function, we can get an adjacency matrix $G^{t(l)}$, consisting of binary values, representing a directed graph. If the element $g_{ij}^{t(l)}$) in $G^{t(l)}$ is 1, the j-th agent will send a message to i-th agent. Otherwise ($g_{ij}^{t(l)} = 0$), the j-th agent will not send any message to i-th agent.

4.3 The Message Processor

The Message Processor helps agents integrate messages properly for apprehension and intelligent decision making. As a black-box, it takes in the encoded messages, $\{m_i^{t(0)}\}_1^N$, and the graphs generated by the Scheduler, $G^{t(1)}\cdots, G^{t(L)}$, and outputs the processed messages, $\{m_i^{t(L)}\}_1^N$. We represent the Message Processor in Equation 8.

$$\{m_i^{t(L)}\}_1^N = f_{MP}\left(m_1^{t(0)}, \cdots, m_N^{t(0)}, G^{t(1)}, \cdots, G^{t(L)}\right) \tag{8}$$

As stated in Section 4.1, the Message Processor consists of L Sub-Processors, each producing a set of encoded messages, $\{m_i^{t(l)}\}_1^N$. A Sub-Processor, for a single round of communication, includes a GAT layer receiving messages, $\{m_i^{t(l-1)}\}_1^N$, from all agents and the adjacency matrix, $G^{t(l)}$, as input. The Sub-Processor will help an agent process and aggregate the messages it receives. The calculation of the attention coefficient in the GAT layer is displayed in Equation 9.

$$\alpha_{ij}^P = \frac{g_{ij}^{t(l)} exp\left(LeakyReLU\left((a_P^{(l)})^\top[W_P^{(l)} m_i^{t(l-1)}||W_P^{(l)} m_j^{t(l-1)}]\right)\right)}{\sum_{k=1}^N g_{ik}^{t(l)} exp\left(LeakyReLU\left((a_P^{(l)})^\top[W_P^{(l)} m_i^{t(l-1)}||W_P^{(l)} m_k^{t(l-1)}]\right)\right)} \tag{9}$$

Here, l is the round of communication, $g_{ij}^{t(l)} \in \{0,1\}$ is a binary value in the adjacency matrix, $G^{t(l)}$, $W_P^{(l)} \in R^{D''\times D}$ is the weight matrix for sub-processor l, and $a_P^{(l)} \in R^{D''}$ is a weight vector for the Sub-Processor. It should be noted that our graphs are capable of a self-loop, where an agent will "send" a message to itself, and use its own message in the integration of received messages. While the calculation of the coefficient for a standard GAT layer is a non-differential operation for the graph, using equation 9 allows us to retain the gradient of $g_{ij}^{t(l)}$. Thus, the Scheduler can preserve the gradient flow for end-to-end training, avoiding the need to design

an extra loss function to train the Scheduler. The output message of sub-processor l can be obtained via Equation 10.

$$m_i^{t(l)} = ELU\left(\sum_{j=1}^{N} \alpha_{ij}^{P} W_P^{t(l)} m_j^{t(l-1)}\right) \tag{10}$$

In practice, we find using the multi-head attention as described in [41], and adding a bias to the output message is beneficial.

4.4 Training

In our experiments, the parameters of the fully-connected layers and LSTM in the policy network are shared across homogeneous agents to improve training efficiency. We employ a multi-threaded synchronous multi-agent policy gradient [37] and utilize an extra value head in the policy network to estimate the value function, $V_\phi(o_i^t)$, at observation o_i^t, which will serve as a state-independent baseline. In addition to optimizing the discounted total reward with policy gradient, the model also minimizes the squared error between the estimated value and the Monte-Carlo estimate. The two loss functions are balanced by a coefficient, β. We define the overall loss function as $\mathcal{L}(\cdot)$ and the policy function denoted as $\pi_\theta(a_i^t|o_i^t)$. Parameters, θ, of the policy and, ϕ, of the value function, share most of their parameters except the parameters in the policy and value heads. Our model is updated via minimizing the loss function displayed in Equation 1.

$$\begin{aligned}\nabla_{\theta,\phi}\mathcal{L}(\theta,\phi) =& \frac{1}{t_{max}} \sum_{i=1}^{N} \sum_{t=1}^{t_{max}} [-\nabla_\theta \log \pi_\theta(a_i^t|o_i^t)(R_i^t - V_\phi(o_i^t)) \\ &+ \beta \nabla_\phi (R_i^t - V_\phi(o_i^t))^2]\end{aligned} \tag{11}$$

Here, R_i^t is the discounted total reward for agent i in an episode, and t_{max} is the number of steps taken within a batch. Different threads in training share the parameters, θ, and ϕ, and calculate their own gradients. The threads synchronously accumulate gradients and update θ and ϕ within each batch. A summary of the training procedure of our multi-agent graph-attention communication model is described in Algorithm 1.

In Algorithm 1, we start by initializing the number of agents alongside several training parameters, including threads, batch size, maximum steps in an episode, and maximum steps in a batch, shown in Step 1. For each update per thread, we start by initializing a set of thread parameters and a replay buffer, D, as displayed in Step 4. After receiving the initial hidden state and observation for each agent (shown in Steps 7 and 8), we can utilize the Scheduler (containing L sub-schedulers) to output adjacency graphs, determining the communication pattern. The Message Processor (containing L sub-processors) can use these graphs and the encoded messages from round zero to produce messages for each agent, as shown in Steps 13 and 14. Once each agent has its selected message inputs, we can determine an action probability distribution from the policy and perform the action sampled from this distribution, shown in Steps 15 and 16. Storing this information in our replay buffer, we can then complete the episode and proceed to compute gradients with Equation 1, as shown in Step 26. Accumulating gradients across threads, we can update our policy and value function, as shown in Steps 29 and 31.

Algorithm 1 Training Multi-Agent Graph-attentIon Communication (MAGIC)

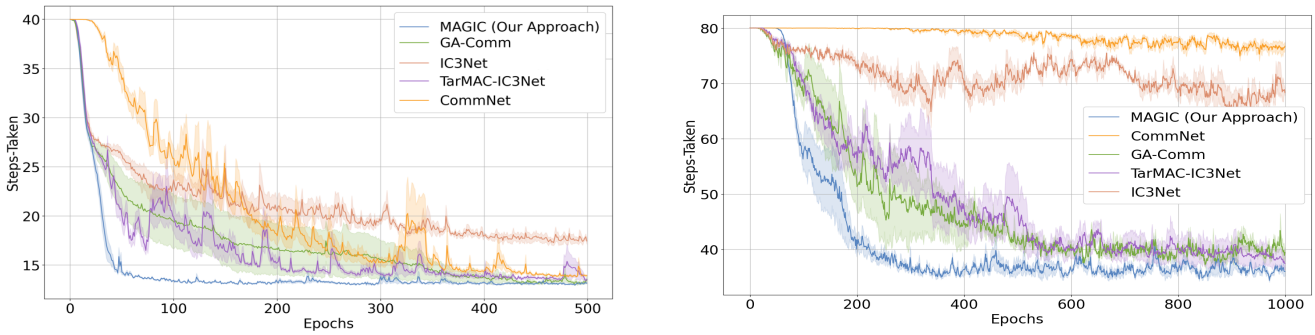
1: Initialize max updates M, agents N, threads L, max steps in an episode T_e, max steps in a batch T_b
2: **for** update = 1 to M **do** $d\theta \leftarrow 0$, $d\phi \leftarrow 0$
3: **for** thread k = 1 to K **do in parallel**
4: Initialize params $\theta_k \leftarrow \theta$, $\phi_k \leftarrow \phi$, buffer D, step-count $t \leftarrow 1$
5: **while** $t < T_b$ **do**
6: Initialize thread step counter $t' \leftarrow 1$
7: Initialize h_i^{t-1}, c_i^{t-1} for each agent i
8: Reset environment and get o_i^t for each agent i
9: **while** $t' < T_e$ and not terminal **do**
10: $h_i^t, c_i^t = LSTM(e(o_i^t), h_i^{t-1}, c_i^{t-1})$ for each agent i
11: $m_i^{t(0)} = e_m(h_i^t)$ for each agent i
12: $\{G^{t(l)}\}_1^L = f_{Sched}\left(m_1^{t(0)}, \cdots, m_N^{t(0)}\right)$
13: $\{m_i^{t(L)}\}_1^N = f_{MP}\left(m_1^{t(0)}, \cdots, m_N^{t(0)}, G^{t(1)}, \cdots, G^{t(L)}\right)$
14: $m_i^t = e'_m(m_i^{t(L)})$ for each agent i
15: Calculate $\pi_{\theta_k}(a_i^t|o_i^t)$ and $V_{\phi_k}(o_i^t)$ for each agent i
16: Perform $a_i^t \sim \pi_{\theta_k}(a_i^t|o_i^t)$ for each agent i
17: Receive r_i^t and o_i^{t+1} for each agent i
18: Store $(o_i^t, a_i^t, \pi_{\theta_k}(a_i^t|o_i^t), V_{\phi_k}(o_i^t), r_i^t, o_i^{t+1})$ in D
19: $t \leftarrow t+1$, $t' \leftarrow t'+1$
20: **end while**
21: **end while**
22: $t_{max} \leftarrow t$
23: **for** $t = t_{max}, t_{max}-1, \cdots, 1$ **do**
24: $R_i^t = 0$ if o_i^{t+1} is terminal else $R_i^t = r_i^t + \gamma R_i^{t+1}$ using D
25: **end for**
26: Calculate $d\theta_k$ and $d\phi_k$ using D with equation 11
27: **end for**
28: **for** thread k = 1 to K **do**
29: Accumulate gradients: $d\theta \leftarrow d\theta + d\theta_k$, $d\phi \leftarrow d\phi + d\phi_k$
30: **end for**
31: Perform update of θ using $d\theta$, and of ϕ using $d\phi$
32: **end for**

5 EVALUATION ENVIRONMENTS

We utilize three domains, including the Predator-Prey [37], Traffic Junction [38] and complex Google Research Football environment [18], to evaluate the utility of proposed communication protocol. Predator-Prey and Traffic Junction are common MARL benchmarks [8, 38]. Google Research Football (GRF) presents a difficult challenge, as it has sparse rewards, stochasticity, and adversarial agents. Additional details are provided in the supplementary.

5.1 Predator-Prey

We utilize the predator-prey environment from Lowe et al. [23]. Here, there are N predators with limited visions searching for a stationary prey. The predators can take actions $up, down, left, right$ or $stay$. We utilize the 'mixed' mode of Predator-Prey in which the predator incurs a reward -0.05 for each time step until the prey is found. An episode is defined as successful if all the predators find the prey before a predefined maximum time limit. We create two levels of difficulty in this environment. The difficulty varies as the grid size, and the number of predators increase, as more coordination is required to achieve success. The corresponding grid sizes and the number of predators are set to 10×10 with 5 predators



(a) Low difficulty Predator-Prey Environment with size 10×10 and 5 agents

(b) High difficulty Predator-Prey Environment with size 20×20, 10 agents

Figure 3: This figure displays the average steps taken to finish an episode as training proceeds in each level of the Predator-Prey environment. The shaded regions represent standard error. A lower value for steps taken on the vertical axis is better.

and 20×20 with 10 predators. We set the maximum steps for an episode (i.e., termination condition) to be 40 and 80, respectively. We define a higher-performing algorithm in this domain as one that minimizes the average steps to complete an episode.

5.2 Traffic Junction

The second domain we utilize is the Traffic Junction environment. This environment, composed of intersecting routes and cars (agents) with limited vision, requires communication to avoid collisions. Cars enter the traffic junction with a probability p_{arrive}. The maximum number of cars crossing a junction at a specific time is denoted as N_{max}, which varies across difficulty levels. A car occupies one cell at a time step and can take action "gas" or "brake" on its route. Two cars collide if they are in the same location, resulting in a reward of -10 for each car. The simulation terminates once all agents reach the end of its route or if the time surpasses the predefined timeout parameter. There is a time penalty -0.01τ at each time step, where τ is the number of time steps that have passed since the agent's entry. An episode is considered successful if there are no collisions within the episode.

We validate our algorithm on two difficulty levels. The simpler level consists of two, one-way roads on a 7×7 grid. There are two arrival points and two possible routes for each arrival point, and there are at most five agents at an intersection ($N_{max} = 5$, $p_{arrive} = 0.3$). For the more difficult level, the junction consists of two, two-way roads on a 14×14 grid with four arrival points, each with three different routes. Here, there are at most ten agents at a junction ($N_{max} = 10$, $p_{arrive} = 0.2$). In this domain, we seek to maximize the average success rate (i.e., no collisions within an episode).

5.3 Google Research Football

Our final domain of Google Research Football [18] presents a challenging, mixed cooperative-competitive, multi-agent scenario with high stochasticity and sparse rewards. Google Research Football (GRF) is a physics-based 3D soccer simulator for reinforcement learning. This last domain presents an additional challenge as there are opponent artificial agents (AIs), significantly increasing the complexity of the state-action space. GRF provides 19 actions including moving actions, kicking actions, and other actions such as dribbling, sliding and sprint. GRF provides several pre-defined reward signals, consisting of a scoring and a penalty box proximity reward. The penalty box proximity reward is shaped to push attackers to move forward towards certain locations. Many MARL frameworks have required these highly shaped rewards functions to perform well [18]. However, we choose to use only the scoring reward to verify the ability for our algorithm and baselines to function in a high-complexity stochastic domain with sparse rewards. **Accordingly, the only reward all agents will receive in our evaluation is +1 when scoring a goal.** The termination criterion is the team scoring, ball out of bounds, or possession change. We evaluate algorithms in the football academy scenario 3 vs. 2, where we have 3 attackers vs. 1 defender, and 1 goalie. The three offending agents are controlled by the MARL algorithm, and the two defending agents are controlled by a built-in AI. We find that utilizing a 3 vs. 2 scenario challenges the robustness of MARL algorithms to stochasticity and sparse rewards. In this domain, we seek to maximize the average success rate (i.e., a goal is scored) and minimize the average steps taken to complete an episode, thereby scoring a goal in the shortest amount of time. As we show in the section 6.3 that our method outperforms all prior state-of-the-art baselines.

6 RESULTS AND DISCUSSION

In this section, we evaluate the performance of our proposed method on three environments, including Predator-Prey [37], Traffic Junction [38], and Google Research Football [18]. We benchmark our approach against a variety of state-of-the-art communication-based MARL baselines, including CommNet [38], IC3Net [37], GA-Comm [22], and TarMAC-IC3Net [8]. We implement our method and baselines on each task, averaging over three random seeds. We provide additional training hyperparameters within the supplementary. Following an analysis of performance, we evaluate MAGIC's communication efficiency, concluding MAGIC presents a new state-of-the-art in both performance and efficiency in MARL.

6.1 Predator-Prey

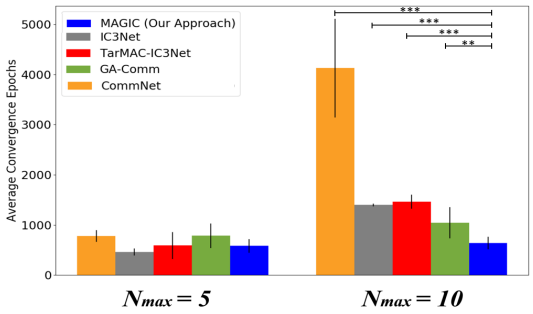


Figure 4: This figure displays the average number of epochs for convergence in Traffic Junction with standard error bars.

Figure 3 depicts the average steps taken for the predators to locate the prey. In both the five- and ten-agent cases, our method converges faster and can achieve better performance than the baselines. Our method converges 62% faster than the next-quickest baseline while still achieving the highest performance. Figure 3 shows that as the number of agents increases, some baselines (CommNet and IC3Net) are unable to learn a coherent policy. Table 1 shows the results of average steps taken to reach the prey at convergence. While approaches such as GA-Comm and TarMAC-IC3Net learn competitive policies for the five agent case, these benchmarks perform much worse than our algorithm in the ten agent case, suggesting a lack of scalability.

Table 1: This table presents the number of steps taken to complete an episode at convergence in Predator-Prey.

Method	10×10, 5 agents	20×20, 10 agents
MAGIC (Our Approach)	**12.733 ± 0.029**	**32.740 ± 0.162**
CommNet [38]	13.146 ± 0.033	73.510 ± 0.725
IC3Net [37]	15.920 ± 0.320	61.566 ± 2.871
TarMAC-IC3Net [8]	13.313 ± 0.112	36.077 ± 0.994
GA-Comm [22]	12.991 ± 0.137	35.752 ± 0.160

6.2 Traffic Junction

We evaluate our method in the Traffic Junction domain for the cases of a maximum of 5 agents and 10 agents at a junction. Table 2 shows the success rate (i.e., no collision in an episode) for each method at convergence in Traffic Junction. Our algorithm achieves near-perfect performance after convergence, widely outperforming all benchmarks in its success rate. Figure 4 depicts the average number of epochs taken to converges for each method. Our method maintains quick convergence as the number of agents increase. However, several of the benchmarks experience a slowdown in their convergence rate. These results show that our method learns a better policy and effectively scale with the number of agents.

6.3 Google Research Football

Lastly, we evaluate our algorithm and several state-of-the-art MARL baselines in the GRF environment. The results presented provide

Table 2: This table presents the success rate at convergence in Traffic Junction.

Method	7×7, $N_{max} = 5$, $p_{arrive} = 0.3$	14×14, $N_{max} = 10$, $p_{arrive} = 0.2$
MAGIC (Our Approach)	**99.9 ± 0.1 %**	**99.9 ± 0.1 %**
CommNet [38]	98.8 ± 0.8%	97.0 ± 0.4%
IC3Net [37]	97.7 ± 1.4%	96.1 ± 1.1%
TarMAC-IC3Net [8]	84.5 ± 4.7%	96.0 ± 1.0%
GA-Comm [22]	95.9 ± 0.1%	97.1 ± 1.1%

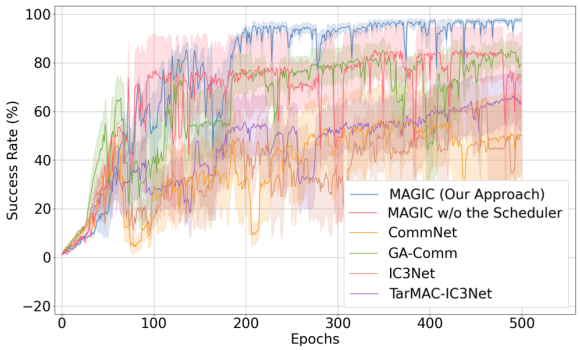


Figure 5: This figure displays the success Rate in GRF as training proceeds. As shown, our method achieves the highest performance, acheiving near-perfect success at scoring.

Table 3: This table displays the success rate and average steps taken to finish an episode in GRF.

Method	Success Rate	Steps Taken
MAGIC (Our Approach)	**98.5 ± 0.9%**	**34.121 ± 1.278**
Ours (without the Scheduler)	85.7 ± 9.9%	37.954 ± 4.774
CommNet [38]	59.6 ± 14.3%	40.906 ± 2.872
IC3Net [37]	68.0 ± 8.5%	42.550 ± 1.555
TarMAC-IC3Net [8]	68.1 ± 10.1%	42.578 ± 3.019
GA-Comm [22]	89.5 ± 4.0%	39.885 ± 2.083

some insight into each algorithm's ability to handle stochasticity, sparse rewards, and a high-complexity state-action space. We utilize the scoring success rate as our metric of evaluation. In this section, we add an additional ablation study, testing our method without the use of the Scheduler. This result signifies the importance of determining "when" and "whom" to communicate with.

Figure 5 displays the success rate for offending agents to score as the training proceeds. Our method converges to a higher-performing policy than all the baselines. While utilizing the Scheduler may adversely affect early performance, it leads to a significant improvement in the performance of our method. In our settings, although each agent only has local observations, it is able to completely observe the state space. MAGIC without the ability to utilize a scheduler performs poorly. It is interesting to note that even with unlimited vision, utilizing a complete graph for communication

Table 4: This table displays the communication efficiency across algorithms. We compute this as the performance improvement obtained when using communication divided by the graph density. As shown, our approach has the highest efficiency.

Method	Graph Density	Avg. Steps w/ Comms	Performance Improvement	$\frac{\text{Improvement}}{\text{Density}}$
MAGIC (Our Approach)	**0.644**	**8.504**	**7.562**	**11.743**
CommNet [38]	0.667	9.216	6.455	9.681
IC3Net [37]	0.638	9.208	6.421	10.058
TarMAC-IC3Net[8]	0.856	9.376	5.958	6.956
GA-Comm [22]	0.514	9.334	5.868	11.391

performs much worse than utilizing a scheduler that gives precise and targeted communication. Table 3 displays the success rate and average steps taken to finish an episode at convergence. Our method has a success rate of 98.5%, which is approximately a 10.1% improvement over the next-best baseline of GA-Comm. Furthermore, our method has approximately a 19x lower average variance than others at convergence. Even though we do not have rewards or time penalties on finishing an episode, and the discount factor is set to 1, we find our learned policy can still score within a short time. Our method achieves the lowest number of average steps, scoring 16.9% more quickly than the closest benchmark of GA-Comm. As we have have outperformed all benchmarks within the previous two domains and in the complex GRF environment, we conclude that our method of "Multi-Agent Graph-attentIon Communication" (MAGIC) is high-performing, scales well to the number of agents, robust to stochasticity, and performs well with sparse rewards.

6.4 Communication Efficiency

We present an analysis of the communication efficiency of our method in Table 4. Communicating efficiently can save resources and allow for messages to be processed more easily. To gauge the efficiency, we utilize the performance improvement due to communication and divide the communication graph density. The graph densities are determined using the sparsity of the adjacency matrix. A fully-connected graph corresponds to a density of 1, and a graph with no connections corresponds to a density of 0. We perform this analysis within a Predator-Prey domain of grid size $5x5$ with 3 predators, where a performance improvement refers to a reduction in steps. To obtain the performance improvement due to communication, we evaluate a communication-blocked variant of each method and compare the performance to the method itself. All methods use the same message size and one round of communication, as some baselines do not support multi-round communication. Utilizing the performance improvement due to communication divided by the graph density as our metric for communication efficiency, we see that MAGIC is the most efficient. MAGIC communicates 23.2% more efficiently than the average baseline while also achieving the highest performance within this domain. Alongside achieving state-of-the-art performance, we are able to most efficiently send and receive messages, displaying the strength of our method.

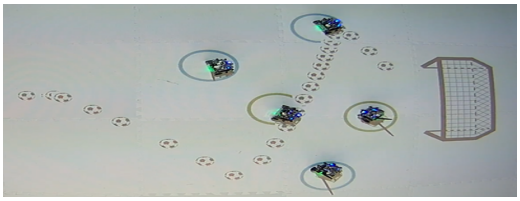


Figure 6: This figure displays a demonstration of our algorithm on physical robots on the Robotarium platform. The display shows a 3 vs. 2 soccer scenario, with blue agents as the attackers, and red agents as defenders.

6.5 Discussion

Across multiple test domains, we set a new state-of-the-art in MARL performance, outperforming baselines, including CommNet [38], IC3Net [37], GA-Comm [22], and TarMAC-IC3Net [8]. Across our domains, we achieve an average 5.6% improvement in steps taken (Predator-Prey), average 2.0% improvement in success rate (Traffic Junction), 10.1% improvement in success rate (GRF), and 16.8% improvement in steps taken (GRF) compared to the closest benchmark which varies across domain. MAGIC is also the most consistent method, achieving the top result as the number of agents and state space increase. The strong performance improvement we achieve in GRF suggests our approach is better able to scale to high-dimension state-action spaces while effectively handling stochasticity and sparse rewards. Additionally, MAGIC's communication protocol is the most efficient communication mechanism across our baselines. MAGIC achieves the highest ratio of performance improvement to communication graph density, outperforming the closest benchmark by 6.5%. By outperforming baselines in both communication efficiency and performance, we set a new state-of-the-art in MARL.

7 PHYSICAL ROBOT DEMONSTRATION

We present a demonstration of our algorithm in a similar 3-vs.-2 soccer scenario on physical robots in the Robotarium, a remotely accessible swarm robotics research platform Pickem et al. [28]. This demonstration displays the feasibility of trajectories produced by our MARL algorithm. We present a depiction of MAGIC's deployed trajectory in Figure 6. A video is attached in the supplementary.

8 CONCLUSION

In this paper, we propose a novel, end-to-end-trainable, graph-attention communication protocol, MAGIC, that utilizes a Scheduler to solve the problems of when to communicate and whom to address messages to, and a Message Processor to integrate and process messages. We evaluate our method and baselines in several environments, including Predator-Prey, Traffic Junction, and the more complex Google Research Football, achieving state-of-the-art performance. In GRF, we achieve a 98.5%, near-perfect success rate in scoring, while most baselines struggle to reach 70%. Not only does MAGIC produce state-of-the-art results, MAGIC is able to converge 62% faster than the next-quickest baseline, and communicates 23.2% more efficiently than the average baseline. Finally, we demonstrate feasibility of MAGIC trajectories on a physical robot testbed.

ACKNOWLEDGMENTS

If you wish to include any acknowledgments in your paper (e.g., to people or funding agencies), please do so using the 'acks' environment. Note that the text of your acknowledgments will be omitted if you compile your document with the 'anonymous' option.

REFERENCES

[1] Dhaval Adjodah, Dan Calacci, Abhimanyu Dubey, Anirudh Goyal, Peter Krafft, Esteban Moro, and Alex Pentland. 2019. Communication topologies between learning agents in deep reinforcement learning. *arXiv preprint arXiv:1902.06740* (2019).

[2] Kavosh Asadi, Cameron S. Allen, Melrose Roderick, Abdel rahman Mohamed, G. Konidaris, and M. Littman. 2017. Mean Actor Critic. *ArXiv* abs/1709.00503 (2017).

[3] M. Babaeizadeh, I. Frosio, S. Tyree, Jason Clemons, and J. Kautz. 2017. Reinforcement Learning through Asynchronous Advantage Actor-Critic on a GPU. In *ICLR*.

[4] In-Chang Baek and Kyung-Joong Kim. 2019. Efficient Multi-Agent Reinforcement Learning Using Clustering for Many Agents.

[5] Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Dębiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, et al. 2019. Dota 2 with large scale deep reinforcement learning. *arXiv preprint arXiv:1912.06680* (2019).

[6] L. Busoniu, R. Babuka, and B. D. Schutter. 2008. A Comprehensive Survey of Multiagent Reinforcement Learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)* 38 (2008), 156–172.

[7] N. Cooke, J. Gorman, Christopher W. Myers, and J. Duran. 2013. Interactive Team Cognition. *Cognitive science* 37 2 (2013), 255–85.

[8] Abhishek Das, Théophile Gervet, Joshua Romoff, Dhruv Batra, Devi Parikh, Michael Rabbat, and Joelle Pineau. 2018. TarMAC: Targeted multi-agent communication. *arXiv preprint arXiv:1810.11187* (2018).

[9] Jakob Foerster, Ioannis Alexandros Assael, Nando De Freitas, and Shimon Whiteson. 2016. Learning to communicate with deep multi-agent reinforcement learning. In *Advances in neural information processing systems*. 2137–2145.

[10] Jakob N Foerster, Gregory Farquhar, Triantafyllos Afouras, Nantas Nardelli, and Shimon Whiteson. 2018. Counterfactual multi-agent policy gradients. In *Thirty-Second AAAI Conference on Artificial Intelligence*.

[11] Shariq Iqbal and Fei Sha. 2018. Actor-attention-critic for multi-agent reinforcement learning. *arXiv preprint arXiv:1810.02912* (2018).

[12] Eric Jang, Shixiang Gu, and Ben Poole. 2016. Categorical reparameterization with gumbel-softmax. *arXiv preprint arXiv:1611.01144* (2016).

[13] Jiechuan Jiang, Chen Dun, and Zongqing Lu. 2018. Graph convolutional reinforcement learning for multi-agent cooperation. *arXiv preprint arXiv:1810.09202* 2, 3 (2018).

[14] Jiechuan Jiang and Zongqing Lu. 2018. Learning attentional communication for multi-agent cooperation. In *Advances in neural information processing systems*. 7254–7264.

[15] Daewoo Kim, Sangwoo Moon, David Hostallero, Wan Ju Kang, Taeyoung Lee, Kyunghwan Son, and Yung Yi. 2019. Learning to schedule communication in multi-agent reinforcement learning. *arXiv preprint arXiv:1902.01554* (2019).

[16] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).

[17] Vijay R Konda and John N Tsitsiklis. 2000. Actor-critic algorithms. In *Advances in neural information processing systems*. 1008–1014.

[18] Karol Kurach, Anton Raichuk, Piotr Stańczyk, Michał Zając, Olivier Bachem, Lasse Espeholt, Carlos Riquelme, Damien Vincent, Marcin Michalski, Olivier Bousquet, et al. 2019. Google research football: A novel reinforcement learning environment. *arXiv preprint arXiv:1907.11180* (2019).

[19] Sheng Li, J. Gupta, Peter Morales, R. Allen, and M. Kochenderfer. 2020. Deep Implicit Coordination Graphs for Multi-agent Reinforcement Learning. *ArXiv* abs/2006.11438 (2020).

[20] S. Li, Yi Wu, Xinyue Cui, Honghua Dong, F. Fang, and S. Russell. 2019. Robust Multi-Agent Reinforcement Learning via Minimax Deep Deterministic Policy Gradient. In *AAAI*.

[21] Michael L Littman. 1994. Markov games as a framework for multi-agent reinforcement learning. In *Machine learning proceedings 1994*. Elsevier, 157–163.

[22] Yong Liu, Weixun Wang, Yujing Hu, Jianye Hao, Xingguo Chen, and Yang Gao. 2019. Multi-Agent Game Abstraction via Graph Attention Neural Network. *arXiv preprint arXiv:1911.10715* (2019).

[23] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, OpenAI Pieter Abbeel, and Igor Mordatch. 2017. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems*. 6379–6390.

[24] Aleksandra Malysheva, Tegg Taekyong Sung, Chae-Bong Sohn, Daniel Kudenko, and Aleksei Shpilman. 2018. Deep multi-agent reinforcement learning with relevance graphs. *arXiv preprint arXiv:1811.12557* (2018).

[25] Chigozie Nwankpa, W. Ijomah, A. Gachagan, and S. Marshall. 2018. Activation Functions: Comparison of trends in Practice and Research for Deep Learning. *ArXiv* abs/1811.03378 (2018).

[26] Afshin Oroojlooyjadid and Davood Hajinezhad. 2019. A Review of Cooperative Multi-Agent Deep Reinforcement Learning. *ArXiv* abs/1908.03963 (2019).

[27] Peng Peng, Ying Wen, Yaodong Yang, Quan Yuan, Zhenkun Tang, Haitao Long, and Jun Wang. 2017. Multiagent bidirectionally-coordinated nets: Emergence of human-level coordination in learning to play starcraft combat games. *arXiv preprint arXiv:1703.10069* (2017).

[28] Daniel Pickem, E. Squires, and M. Egerstedt. 2016. The Robotarium : An open , remote-access , multi-robot laboratory.

[29] Heechang Ryu, Hayong Shin, and Jinkyoo Park. 2019. Multi-Agent Actor-Critic with Hierarchical Graph Attention Network. *arXiv preprint arXiv:1909.12557* (2019).

[30] E. Salas, N. Cooke, and M. Rosen. 2008. On Teams, Teamwork, and Team Performance: Discoveries and Developments. *Human Factors: The Journal of Human Factors and Ergonomic Society* 50 (2008), 540 – 547.

[31] E. Salas, T. Dickinson, Sharolyn A. Converse, and S. Tannenbaum. 1992. Toward an understanding of team performance and training.

[32] Mikayel Samvelyan, Tabish Rashid, C. S. Witt, Gregory Farquhar, Nantas Nardelli, Tim G. J. Rudner, Chia-Man Hung, P. Torr, Jakob N. Foerster, and S. Whiteson. 2019. The StarCraft Multi-Agent Challenge. In *AAMAS*.

[33] S. Scardapane, Indro Spinelli, and P. D. Lorenzo. 2020. Distributed Graph Convolutional Networks. *ArXiv* abs/2007.06281 (2020).

[34] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. 2008. The graph neural network model. *IEEE Transactions on Neural Networks* 20, 1 (2008), 61–80.

[35] Junjie Sheng, Xiangfeng Wang, Bo Jin, Junchi Yan, Wenhao Li, Tsung-Hui Chang, Jun Wang, and Hongyuan Zha. 2020. Learning Structured Communication for Multi-agent Reinforcement Learning. *arXiv preprint arXiv:2002.04235* (2020).

[36] D. Silver, G. Lever, N. Heess, T. Degris, Daan Wierstra, and M. Riedmiller. 2014. Deterministic Policy Gradient Algorithms. In *ICML*.

[37] Amanpreet Singh, Tushar Jain, and Sainbayar Sukhbaatar. 2018. Learning when to communicate at scale in multiagent cooperative and competitive tasks. *arXiv preprint arXiv:1812.09755* (2018).

[38] Sainbayar Sukhbaatar, Rob Fergus, et al. 2016. Learning multiagent communication with backpropagation. In *Advances in neural information processing systems*. 2244–2252.

[39] Richard S Sutton, David A McAllester, Satinder P Singh, and Yishay Mansour. 2000. Policy gradient methods for reinforcement learning with function approximation. In *Advances in neural information processing systems*. 1057–1063.

[40] Güliz Tokadli and Michael C. Dorneich. 2019. Interaction Paradigms: from Human-Human Teaming to Human-Autonomy Teaming. *2019 IEEE/AIAA 38th Digital Avionics Systems Conference (DASC)* (2019), 1–8.

[41] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).

[42] Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. 2019. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature* 575, 7782 (2019), 350–354.

[43] Rundong Wang, Xu He, Runsheng Yu, Wei Qiu, Bo An, and Zinovi Rabinovich. 2019. Learning Efficient Multi-agent Communication: An Information Bottleneck Approach. *arXiv preprint arXiv:1911.06992* (2019).

[44] Zheyuan Wang and Matthew Craig Gombolay. 2019. Learning to Dynamically Coordinate Multi-Robot Teams in Graph Attention Networks. *ArXiv* abs/1912.02059 (2019).

[45] Christopher JCH Watkins and Peter Dayan. 1992. Q-learning. *Machine learning* 8, 3-4 (1992), 279–292.

[46] Ronald J Williams. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning* 8, 3-4 (1992), 229–256.

[47] Zonghan Wu, Shirui Pan, Fengwen Chen, G. Long, C. Zhang, and P. Yu. 2020. A Comprehensive Survey on Graph Neural Networks. *IEEE transactions on neural networks and learning systems* (2020).